

Machine Learning

Winter 2011/12

Roland Memisevic

Lecture 2, Oct. 24, 2011

Linear Regression

- ▶ Linear regression is one of the most basic and most fundamental concepts in machine learning and statistics.
- ▶ It is a *simple problem* with a *simple solution*.
- ▶ Yet, linear regression allows us to highlight and investigate a variety of basic and central concepts in ML, which will be recurring throughout the lecture:
 1. Learning can be viewed as an optimization task.
 2. Learning can be viewed as a maximum likelihood task.
 3. Learning can be viewed as Bayesian inference.
 4. Learning is affected by the Bias/Variance tradeoff.
- ▶ Because of its simplicity, linear regression is also used in extremely many practical tasks.

Outline

- ▶ Linear regression
- ▶ Learning as optimization and Stochastic gradient descent
- ▶ Learning as (regularized) Maximum Likelihood
- ▶ The Bias-Variance Decomposition
- ▶ A first glimpse at Bayesian Learning

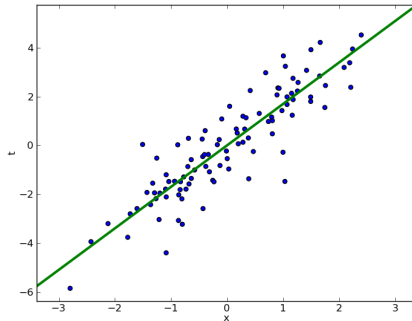
Linear Regression

$$\mathbf{x} \rightarrow \mathbf{t}$$

- ▶ We are given two *real-valued* observations $\mathbf{x}$ and $\mathbf{t}$.
- ▶ Task: *Learn to predict $\mathbf{t}$ from $\mathbf{x}$.*
- ▶ So this is a *supervised learning* problem.

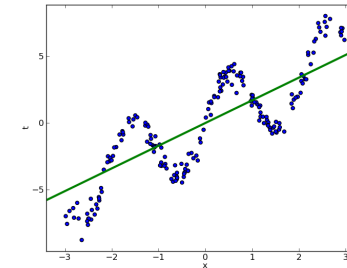
Linear Regression in 1-d

- ▶ When both inputs and outputs are scalars (1-d), this is easy to visualize:



- ▶ Linear regression assumes that there is a *linear* dependency between x and t .

Noise vs. dependencies we don't care about



- ▶ Even if we *do not actually believe this*, we can still use linear regression, and model just the *linear trend* in data, declaring any non-linear dependencies as noise!
- ▶ (Note that the noise may be highly non-Gaussian.)

Linear Regression in 1-d

- ▶ For 1-d inputs and 1-d outputs, the linear regression model takes the form

$$y(x) = w_0 + w_1x$$

- ▶ For D-dimensional inputs and 1-d outputs it takes the form

$$y(\mathbf{x}) = w_0 + w_1x_1 + w_2x_2 + \dots + w_Dx_D (= w_0 + \mathbf{w}^T\mathbf{x})$$

- ▶ For K-dimensional outputs, we simply have one model *in each dimension* of $\mathbf{y}$:

$$\begin{pmatrix} y_1(\mathbf{x}) \\ \vdots \\ y_K(\mathbf{x}) \end{pmatrix} = \begin{pmatrix} y_1(\mathbf{x}) = w_{10} + w_{11}x_1 + w_{12}x_2 + \dots + w_{1D}x_D \\ \vdots \\ y_K(\mathbf{x}) = w_{K0} + w_{K1}x_1 + w_{K2}x_2 + \dots + w_{KD}x_D \end{pmatrix}$$

The bias term

- ▶ w_0 is called “bias term” or “intercept parameter”, because it allows us to move the linear model along the y -axis (or axes).
- ▶ It is easy, and common, to *eliminate* it (formally) from our models, and thereby to simplify calculations:
- ▶ Simply replace the input vector:

$$\begin{pmatrix} x_1 \\ \vdots \\ x_D \end{pmatrix} \rightarrow \begin{pmatrix} 1 \\ x_1 \\ \vdots \\ x_D \end{pmatrix}$$

- ▶ Now any pure linear model will *implicitly* contain the bias term. So we can write, for example, $\mathbf{w}^T\mathbf{x}$ (drop w_0).

Least squares

- ▶ We now consider the task of learning the parameters (aka “weights”) of the model. We shall restrict our attention to 1-dimensional outputs at first.
- ▶ To learn the parameters from a training set

$$\mathcal{D} = \{(\mathbf{x}_n, t_n)\}_{n=1}^N$$

we minimize the total *sum-of-squares error*:

$$E(\mathbf{w}; \mathcal{D}) = \frac{1}{2} \sum_{n=1}^N (t_n - \mathbf{w}^T \mathbf{x}_n)^2$$

with respect to $\mathbf{w}$.

- ▶ Many other choices of error function are possible. But the sum-of-squares error makes the math easy.

Least squares

- ▶ We can write this more compactly with the following definitions: Let $\mathbf{t}$ be the vector of “stacked” outputs

$$\mathbf{t} = \begin{pmatrix} t_1 \\ \vdots \\ t_N \end{pmatrix}$$

and let

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_1 \\ \vdots \\ \mathbf{x}_N \end{pmatrix}$$

be a matrix containing one input vector per row. With this we can rewrite the solution as:

The normal equations

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{t}$$

Least squares

- ▶ To optimize with respect to $\mathbf{w}$, we differentiate:

$$\frac{\partial E}{\partial \mathbf{w}} = - \sum_{n=1}^N (t_n - \mathbf{w}^T \mathbf{x}_n) \mathbf{x}_n^T$$

- ▶ Setting the derivative to zero:

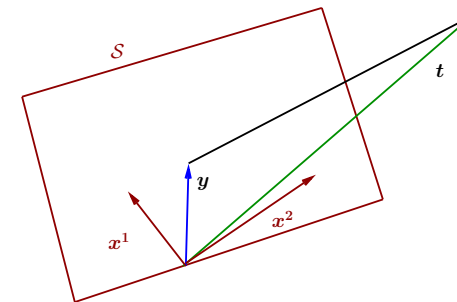
$$0 = - \sum_{n=1}^N t_n \mathbf{x}_n^T + \mathbf{w}^T \left(\sum_{n=1}^N \mathbf{x}_n \mathbf{x}_n^T \right)$$

yields the solution

$$\mathbf{w} = \left(\sum_{n=1}^N \mathbf{x}_n \mathbf{x}_n^T \right)^{-1} \left(\sum_{n=1}^N t_n \mathbf{x}_n^T \right)$$

- ▶ (It can be instructional to write down the case for 1-d inputs, if this confuses you)

A geometrical interpretation



- ▶ We can think of the squared error $\sum_n (t_n - \mathbf{w}^T \mathbf{x}_n)^2$ as the squared norm of two N -dimensional vectors $\mathbf{t}$ and $\mathbf{y}$, containing all training-outputs and all model-predictions, respectively.
- ▶ The vector $\mathbf{y} (= \mathbf{X}\mathbf{w})$ has to lie in the space $\mathcal{S}$ spanned by the *columns* $\mathbf{x}^i$ of $\mathbf{X}$.
- ▶ It is exactly the *orthogonal projection* of $\mathbf{t}$ onto $\mathcal{S}$

Multi-dimensional outputs

- ▶ In many applications, we need to predict $K > 1$ variables.
- ▶ Since this amounts to having one model in each dimension, there are no significant changes as compared to the 1-dimensional solution.
- ▶ In particular, we now have a *matrix* $\mathbf{W}$ of parameters, and we minimize the cost

$$E(\mathbf{W}; \mathcal{D}) = \frac{1}{2} \sum_{n=1}^N \|\mathbf{t}_n - \mathbf{W}^T \mathbf{x}_n\|^2$$

- ▶ This leads to:

Learning with gradient descent

- ▶ We have seen that we can think of **learning as optimizing a cost (or “loss”) function**.
- ▶ We will soon see learning tasks, for which we do not have a closed-form solution for the optimization.
- ▶ Also, often data may arrive one point at a time, and we want to make predictions as we go.
- ▶ For all these cases, there is a simple, general approach to learning.
- ▶ It is applicable, whenever we can write the cost function as the *sum over individual training cases*, that is:

$$E = \sum_n E_n$$

for some E_n that depends only on training case n .

Multi-dimensional outputs

The normal equations (multidimensional outputs)

$$\mathbf{W} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{T}$$

where each column of $\mathbf{T}$ contains a different N -vector $\mathbf{t}_k$ of outputs.

- ▶ Thus, the K -dimensional case is like K independent 1-dimensional cases.
- ▶ It is therefore sufficient to restrict our attention to the 1-dimensional case in what follows.

Learning with gradient descent

- ▶ In this case, we can use an iterative learning rule called:

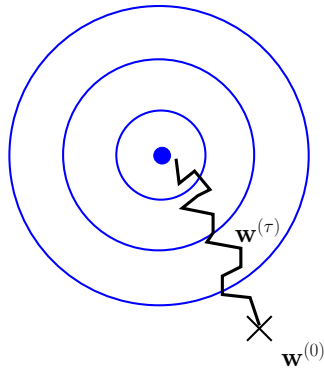
Stochastic gradient descent

$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta \frac{\partial E_n}{\partial \mathbf{w}}$$

- ▶ Here, τ denotes the iteration number.
- ▶ Parameters are initialized to some random starting value $\mathbf{w}^{(0)}$.
- ▶ η is called **learning rate**, and it is typically set to a small real value (such as, $\eta = 0.001$). It may be reduced as learning progresses.
- ▶ In the special case of least squares regression, we get

$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} + \eta (t_n - \mathbf{w}^{(\tau)T} \mathbf{x}_n) \mathbf{x}_n$$

Learning with gradient descent



- ▶ Since the algorithm visits one training-case at a time, it will jitter around an idealized “average path” towards the optimum.

Basis expansion, some 1-d examples

- ▶ Polynomial:

$$\phi_j(x) = x^j \quad j = 1, \dots, M$$

- ▶ Can be easily extended to multi-dimensional inputs.
- ▶ “Gaussian” or “radial” basis-functions:

$$\phi_j(x) = \exp\left(-\left(\frac{x - \mu_j}{2s^2}\right)^2\right)$$

- ▶ Sigmoidal basis functions.
- ▶ Etc.

Non-linear basis functions

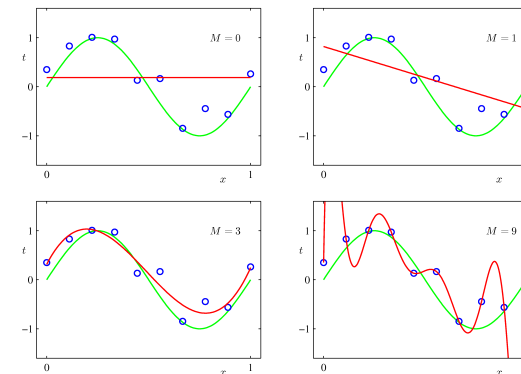
- ▶ There is a very simple way to extend linear regression, such that we can model a *non-linear* relationship between inputs and outputs:
- ▶ **Pre-process** the inputs using a set of **fixed**, non-linear functions.
- ▶ For this end, replace

$$\mathbf{x} \rightarrow \Phi(\mathbf{x})$$

where Φ is some non-linear function, and treat Φ as your new inputs. Everything else stays the same.

- ▶ In particular, the model is still *linear* with respect to the model parameters.
- ▶ This is known as **basis expansion**. Some examples:

Polynomial example



- ▶ This is the polynomial task from lecture 1.
- ▶ Note that the input (Φ)-space has different dimension in each shown case, even though the original input ($\mathbf{x}$)-space is always 1-dimensional.

Gaussian Noise and Maximum likelihood

- ▶ So far, we treated learning as an optimization algorithm.
- ▶ We can obtain a probabilistic interpretation, if we make the following assumption:

$$t = y(\mathbf{x}, \mathbf{w}) + \epsilon$$

- ▶ In words, we assume there is a true, underlying function $y(\mathbf{x}, \mathbf{w})$, and the function values we observe are corrupted by *additive Gaussian noise*.
- ▶ Thus

$$p(t|\mathbf{x}; \mathbf{w}) = \mathcal{N}(t|y(\mathbf{x}, \mathbf{w}), \sigma^2)$$

Gaussian Noise and Maximum likelihood

- ▶ To fit the conditional Gaussian, given training data $\mathcal{D} = \{(\mathbf{x}_n, t_n)\}_{n=1}^N$, we assume all cases were drawn independently from the same distribution (known as **iid**: “**independent and identically distributed**”), so

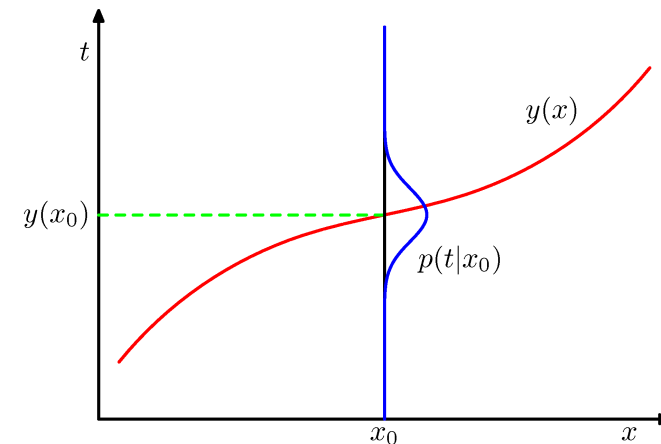
$$p(\mathcal{D}) = \prod_n \mathcal{N}(t_n|y(\mathbf{x}_n, \mathbf{w}), \sigma^2)$$

- ▶ Using monotonicity of the log, we may maximize the *log-probability* (or “-likelihood”):

$$\text{maximize} \quad - \sum_{n=1}^N (t_n - \mathbf{w}^T \mathbf{x}_n)^2 + \text{const.}$$

- ▶ This is equivalent to *minimizing* the least squares cost $E(\mathbf{w}; \mathcal{D})$ we saw before.

Gaussian Noise and Maximum likelihood



Gaussian Noise and Maximum likelihood

- ▶ Maximum likelihood is a general principle (or “recipe”) for fitting the parameters of distributions. We will see more examples throughout the course.
- ▶ Maximum likelihood can be used to fit both unconditional and conditional distributions.
- ▶ Maximum likelihood can be used to fit distributions over both discrete and continuous spaces.
- ▶ Unfortunately, maximum likelihood (and least squares regression) can lead to **overfitting**, especially when the number of training cases is small and the number of input dimensions is large.

Regularized least squares

- ▶ A common way to prevent overfitting is to add a *penalty term* to the objective function:

$$E_{\lambda}(\mathbf{w}, \mathcal{D}) = E(\mathbf{w}, \mathcal{D}) + \lambda E_W(\mathbf{w})$$

- ▶ The most common type of penalty term encourages weights to be small, by penalizing the squared norm of $\mathbf{w}$:

$$E_W(\mathbf{w}) = \frac{1}{2} \mathbf{w}^T \mathbf{w}$$

- ▶ Note that this will automatically discourage the unstable, wiggly behaviour in the polynomial due to large coefficients.
- ▶ This form of regularization is also known as “**weight decay**”.
- ▶ Of course, λ will need to be set using validation data!

The lasso

- ▶ One particularly common alternative is to penalize the 1-norm of the weights:

$$E_W(\mathbf{w}) = \lambda \sum_j |w_j|$$

- ▶ This is known as “**lasso** regression” (Tibshirani, 1996).
- ▶ It is popular because it encourages weights to be **sparse**: Weights will either be far from zero, or be *exactly* zero.
- ▶ So it allows us to answer questions like “Which of a given set of dependencies (eg., *quadratic* vs. *sinusoidal* vs. *linear*) are supported by our data?”

Regularized least squares

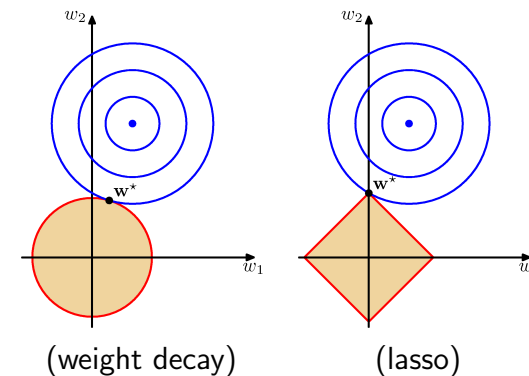
- ▶ It is straightforward to show that the optimal parameters when using weight decay are given by:

Ridge regression

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{t}$$

- ▶ One can use other kinds of regularizer than the squared norm of the weights.
- ▶ But the optimization then typically becomes more difficult.

Weight decay and lasso intuition



- ▶ Optimizing “*objective + weight penalty*” is like optimizing the objective under a *constraint* on the parameters.
- ▶ This shows why the lasso penalty tends to yield sparse solutions, whereas weight decay does not.

The Bias-Variance Decomposition

- Assume, our data is actually stochastic, and it is governed by a joint distribution $p(\mathbf{x}, t)$.
- Then, the best prediction, in terms of squared error, we can ever make on some test-case $\mathbf{x}_n$ will be the *conditional expectation*

$$\mathbb{E}[t|\mathbf{x}_n] = \int t p(t|\mathbf{x}_n) dt$$

- Of course, we can never make that optimal prediction, since we will always have a finite amount of training data.

The Bias-Variance Decomposition

- Let us consider the following thought experiment:
- We re-estimate model parameters $\mathbf{w}$ many times using a different training set $\mathcal{D}$ each time.
- Now consider the average over *choices of dataset* of the squared error on some fixed test-case $\mathbf{x}_n$:

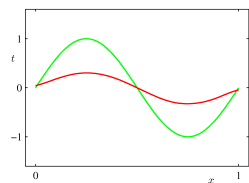
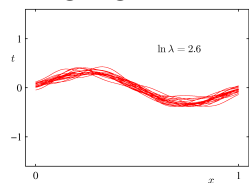
$$\mathbb{E}_{\mathcal{D}}[(y(\mathbf{x}_n; \mathcal{D}) - \mathbb{E}[t|\mathbf{x}_n])^2]$$

This can be written (derivation, Bishop page 149):

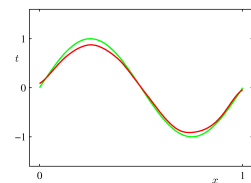
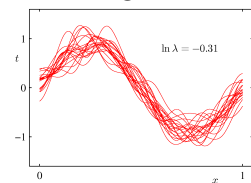
$$\begin{aligned} & (\mathbb{E}_{\mathcal{D}}[y(\mathbf{x}_n; \mathcal{D})] - \mathbb{E}[t|\mathbf{x}_n])^2 + \mathbb{E}_{\mathcal{D}}[(y(\mathbf{x}_n; \mathcal{D}) - \mathbb{E}_{\mathcal{D}}[y(\mathbf{x}_n; \mathcal{D})])^2] \\ & =: (\text{bias})^2 + \text{variance} \end{aligned}$$

The Bias-Variance Decomposition

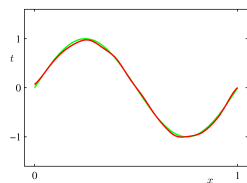
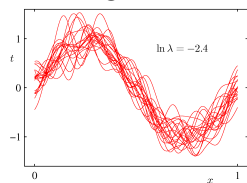
strong regularization



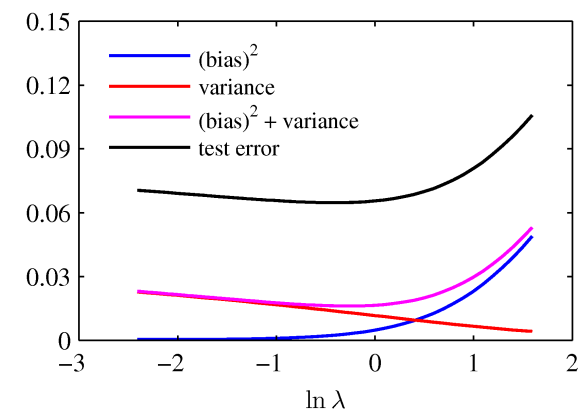
weaker regularization



weak regularization



The Bias-Variance Decomposition



- In practice, to find the right amount of regularization, we need to use validation data.

A first glimpse at Bayesian Learning

- ▶ Maximum likelihood learning amounts to searching for parameters $\mathbf{w}$, such that the likelihood of the training data $p(\mathcal{D}; \mathbf{w})$ becomes maximal.
- ▶ Regularization is used to reduce overfitting.
- ▶ It is possible to eliminate overfitting, in fact *fitting* of models altogether, if we are willing to think of the parameters $\mathbf{w}$ themselves as *random variables*.

A first glimpse at Bayesian Learning

- ▶ We can think of Bayesian modeling as a way to update our *assumptions* $p(\mathbf{w})$ about the model parameters in the presence of data $\mathcal{D}$.
- ▶ Computing the normalizing constant $p(\mathcal{D})$ is often very difficult in practice.
- ▶ Being able to compute it, however, allows us to perform **model comparison**, since it can tell us, how likely the data is under a given *model class*!

A first glimpse at Bayesian Learning

- ▶ For this end, let us re-write the likelihood as a *conditional distribution* $p(\mathcal{D}|\mathbf{w})$ rather than a distribution that is merely *parameterized* by $\mathbf{w}$.
- ▶ Using **Bayes' rule**, we can now write:

$$p(\mathbf{w}|\mathcal{D}) = \frac{p(\mathcal{D}|\mathbf{w})p(\mathbf{w})}{p(\mathcal{D})}$$

- ▶ $p(\mathbf{w}|\mathcal{D})$ is known as **posterior distribution** over $\mathbf{w}$.
- ▶ To compute it, we need to first define $p(\mathbf{w})$, known as the **prior distribution** over $\mathbf{w}$.

An example of using a Gaussian prior on $\mathbf{w}$

